



Digital Design Verification

Lab Manual # 22 – Handling Data Hazards

Release: 1.0

Date: 25-June-2024

NUST Chip Design Centre (NCDC), Islamabad, Pakistan



Copyrights ©, NUST Chip Design Centre (NCDC). All Rights Reserved. This document is prepared by NCDC and is for intended recipients only. It is not allowed to copy, modify, distribute or share, in part or full, without the consent of NCDC officials.

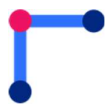
Revision History

Revision Number	Revision Date	Revision By	Nature of Revision	Approved By
1.0	25/06/2024	Ali Aqdas	Complete manual	-
2.0	19/08/2025	Umer Farooq / Fawad Ahmed	Manual Update	



Contents

Contents.....	2
Objective	3
Tools	3
Instructions	3
RISC-V Data and Control Paths.....	4
Pipelining: Data Hazards	4
Data Hazards: Forwarding to EX Stage	4
Data Hazards: Forwarding to MEM Stage	5
Data Hazards: Stalling	6
Hazard Detection and Stalling Implementation	6
Task	8
References	8



Objective

The objective of this lab is to

- Append the design with forwarding and hazard detection

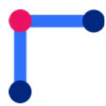
Tools

- Cadence Xcelium

Instructions

The students must design and submit the updated files for evaluation of this lab. **Additional modules may be needed for forwarding and hazard detection.**

./<student-name>/	./rtl/	program_counter.sv inst_mem.sv data_mem.sv reg_file.sv imm_gen.sv alu_logic.sv branch_comp.sv control_unit.sv IF_ID_Stage.sv ID_EX_Stage.sv EX_MEM_Stage.sv MEM_WB_Stage.sv fwd_logic.sv hazard_detection.sv top.sv
	./tb/	program_counter_tb.sv inst_mem_tb.sv data_mem_tb.sv reg_file_tb.sv imm_gen_tb.sv alu_logic_tb.sv branch_comp_tb.sv control_unit_tb.sv IF_ID_tb.sv ID_EX_tb.sv EX_MEM_tb.sv MEM_WB_tb.sv fwd_logic_tb.sv hazard_detection_tb.sv top_tb.sv



RISC-V Data and Control Paths

The pipelined data and control paths designed so far are shown in the figure below.

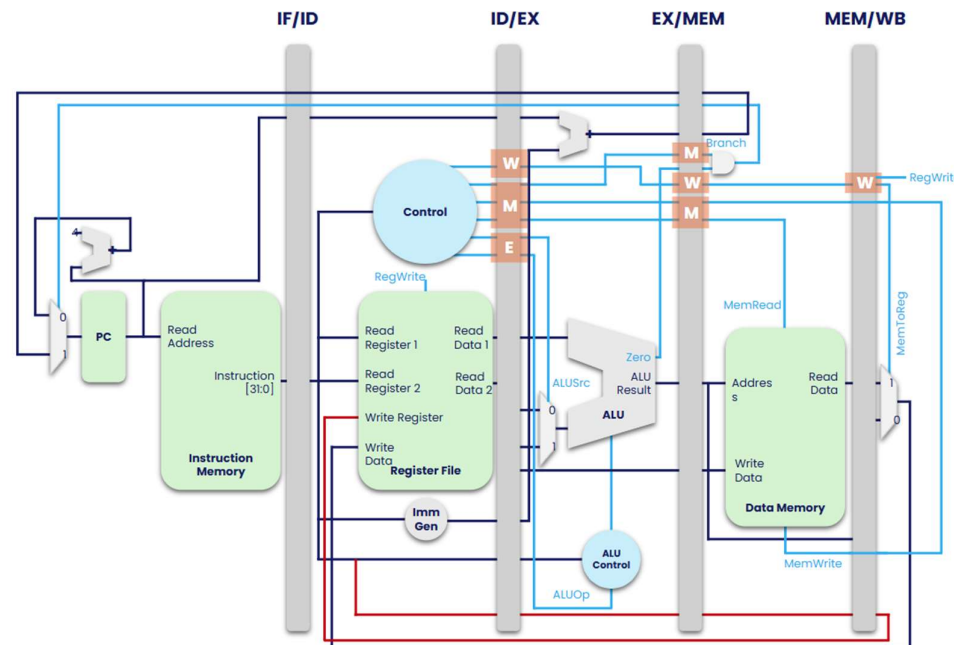


Figure 1: Pipelined Datapath

Pipelining: Data Hazards

In pipelined design, data, control and structural hazards can occur, which may disrupt the operation of processor leading to unexpected behavior. Let us discuss data hazards step-by-step in detail for sake of clarity.

Data Hazards: Forwarding to EX Stage

In the pipelined design, only one data hazard can occur, i.e. Read After Write (RAW).

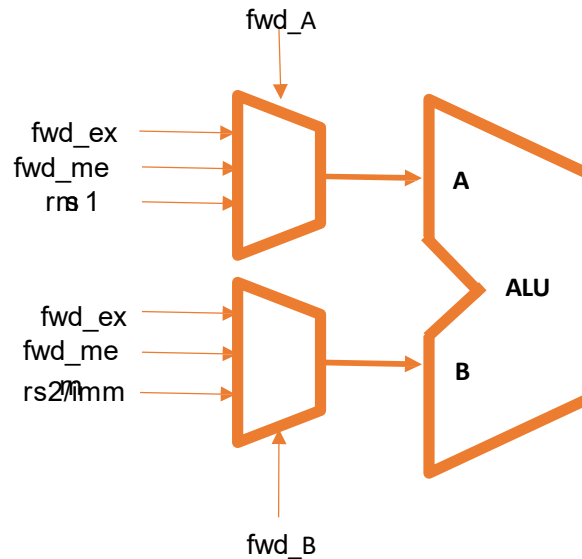
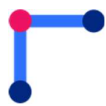
Consider the following assembly codes,

Example 1	Example 2
add x10,x20,x21	add x10,x20,x21
add x11,x22,x23	add x11,x22,x23
add x12,x24,x25	add x12,x24,x25
addi x13,x12,1	addi x13,x11,1

In the first example, the **third instruction** clashes with the **highlighted instruction** due to register **x12**. As the third instruction reaches MEM stage, highlighted instruction reaches EX Stage and requires the updated value of **x12**. **Hence, we need to forward the operand from EX/MEM Pipeline Register to EX Stage.**

A similar case occurs in the second example and the data needs to be forwarded from MEM/WB Register to EX Stage.

The resulting logic could be designed with an additional multiplexor immediately before ALU.

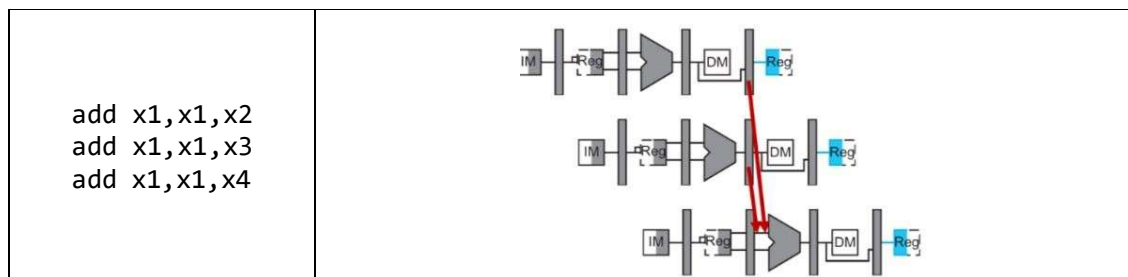


Two forwarding control signals “*fwd_A*” and “*fwd_B*” indicate whether forwarding is needed and from which stage.

The forwarding signals are generated by checking the conditions listed in the table below.

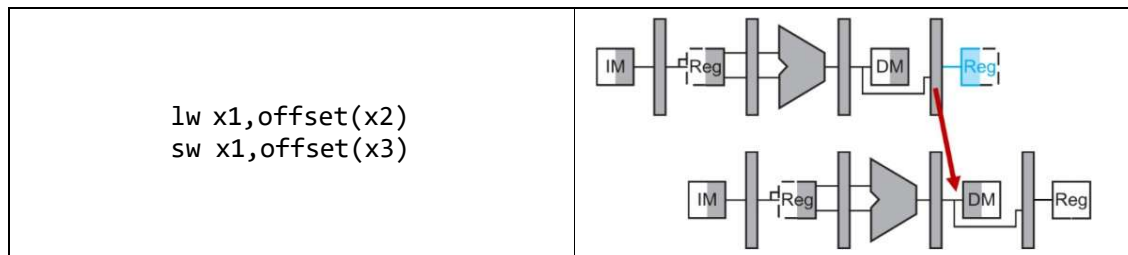
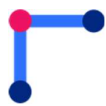
Address Comparison Conditions	Zero Check	RegWrite Check	Additional Conditions
EX/MEM.RegisterRd == ID/EX.RegisterRs1	Rd != x0	EX/MEM.RegWrite=1	NIL
EX/MEM.RegisterRd == ID/EX.RegisterRs2			
MEM/WB.RegisterRd == ID/EX.RegisterRs1		ME/WB.RegWrite=1	EX Hazard Must Not Exist
MEM/WB.RegisterRd == ID/EX.RegisterRs2			

The additional condition required for some hazards needs further explanation for clarification. In the example below, hazards demand forwarded data from both the MEM/WB and EX/MEM Stages. However, only one data can be forwarded to an operand. In that case, we take the most recent data updated by the second instruction, which will be in the EX Stage.



Data Hazards: Forwarding to MEM Stage

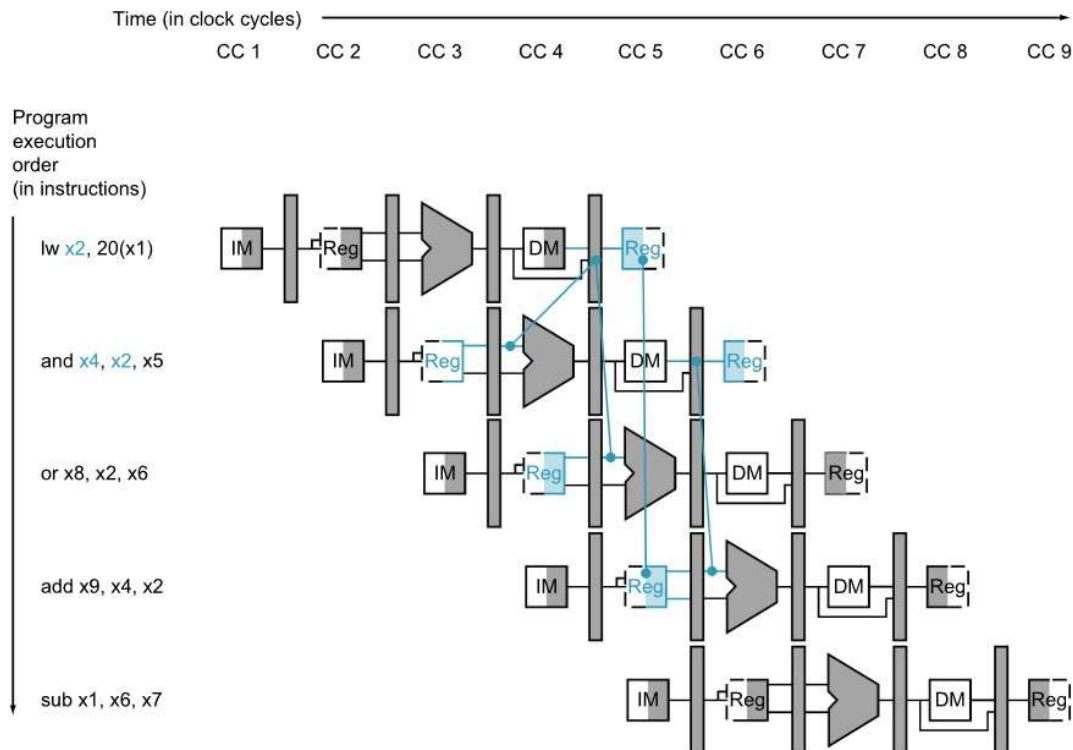
In consecutive load-store instructions, as shown in the example code below.



The data from load stage can be forwarded to the store instruction through multiplexors as before with an additional forwarding logic.

Data Hazards: Stalling

In certain situations, it becomes necessary to stall the pipeline. An example is given in the figure below, where data from load instruction is immediately required by “and” instruction. It is not possible to forward, because the “and” instruction would already have passed through execute stage one cycle before the data from load is available. Hence, we must stall.



Hazard Detection and Stalling Implementation

We must not allow the stalled instruction to make any changes to the state elements in the stages before the stage where instruction is stalled, and in the global state elements like memory and register file.

In the above case, it is required that the following do not change

1. IF/ID Pipeline Registers
2. Program Counter

and the following are written to disable memory and register file write for that instruction.

1. Register Write Signal
2. Memory Write Signal



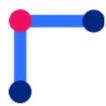
A hazard detection unit checks for potential hazards and generates control signals for stalls. We are required to check the following conditions in the hazard detection unit. If both the conditions are true, then hazard exists, and pipeline must be stalled.

Memory Load Condition	Condition for Correspondence with Source Registers
ID/EX.MemRead	ID/EX.RegisterRd=IF/ID.RegisterRs1 OR ID/EX.RegisterRd=IF/ID.RegisterRs2

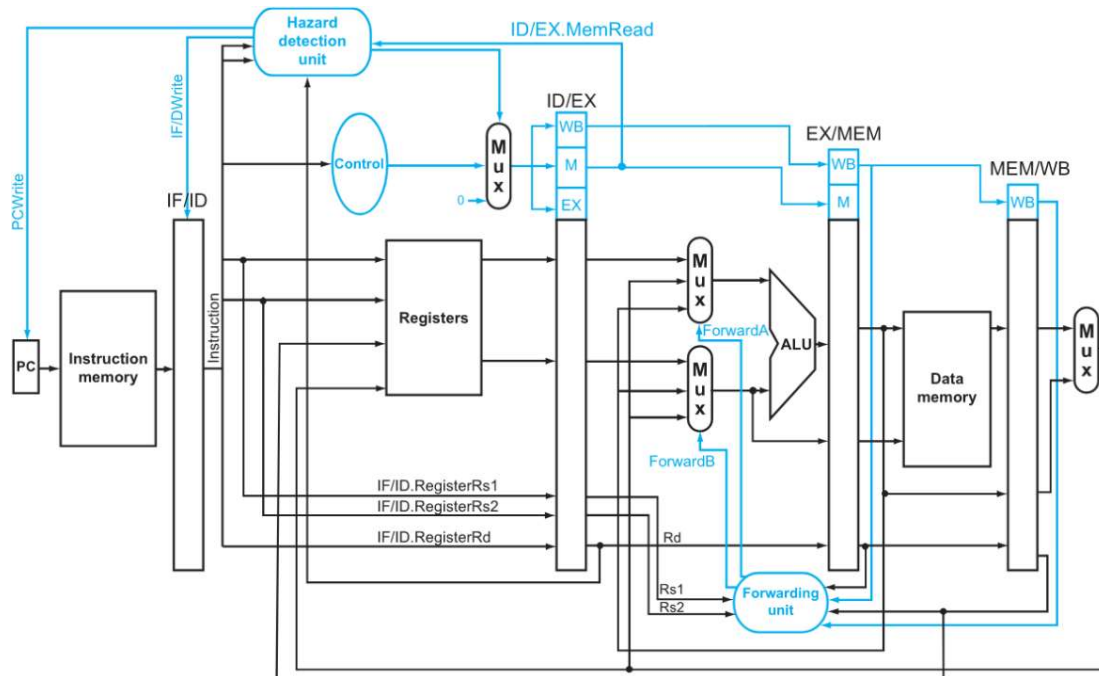
Now, to implement stalls we must modify the IF/ID pipeline register (perhaps by appending a logical circuit), to add a write enable signal, which can control whether it should be written. In the case of a stall, write enable signal is set to **LOW** and data is not updated in the pipeline register.

Additionally, the **RegWrite** and **MemWrite** are set to **LOW** for stalled instruction so that they are not propagated in the pipeline, and memory and register does not change.

To prevent the next instruction from being fetched, a **PCWrite** signal needs to be introduced in the program counter. When **PCWrite** is **LOW**, it must halt the program counter and must not let it increment.



A reference block diagram with both units included is shown in the Figure below.



Task

Append your pipelined datapath, to include forwarding and hazard detection, which will resolve hazards and improve performance.

References

1. Computer Architecture and Organization (RISC-V Edition), David Patterson, John Hennessy
2. University of California, Berkeley (CS61C)